EE627 — Class Summary

April 15, 2026

Prof. Rensheng Wang

Contents

1 From Small-Scale MF to Big-Data MF **3**
1.1 Recap: The Homework 7 Picture . . . 3
1.2 Why That Breaks at Industrial Scale . . . 3
1.3 The Shift . . . 3

2 Big Data: The Four V's **3**

3 Hadoop and HDFS **4**
3.1 Origin and Purpose . . . 4
3.2 Hadoop Distributed File System (HDFS) . . . 4

4 MapReduce **4**
4.1 Programming Model . . . 4
4.2 Why It Scales . . . 4
4.3 Its Limitation — And Why Spark Was Needed . . . 5

5 Apache Spark **5**
5.1 What It Is . . . 5
5.2 Python vs. Scala . . . 5

6 Spark Architecture **5**

7 RDDs: Spark's Core Abstraction **6**
7.1 Definition . . . 6
7.2 Two Ways to Create RDDs . . . 6
7.3 Transformations vs. Actions . . . 6
7.4 Lazy Evaluation . . . 6
7.5 Typical Spark Program Skeleton . . . 6

8 Common Transformations and Actions **7**
8.1 Element-Wise Transformations . . . 7
8.2 Actions . . . 7

9 Pair RDDs (Key/Value) **7**
9.1 Creating a Pair RDD . . . 7
9.2 Useful Pair-RDD Operations . . . 8

9.3 Canonical Example: Word Count . . . 8
9.4 Per-Key Average (from the Spark slides) . . . 8

10 DataFrames in PySpark **8**

11 PySpark ALS for the Final Project **8**
11.1 Environment Setup on Google Colab . . . 8
11.2 Data Format . . . 9
11.3 ALS Pipeline . . . 9
11.4 Why ALS, Not SGD . . . 10
11.5 Data Files Accompanying the Lecture . . . 10

12 Why This Matters for the Ensemble **11**
12.1 Heuristic vs. Collaborative . . . 11
12.2 Diversity for Ensembling . . . 11

13 Key Reference **11**

1 From Small-Scale MF to Big-Data MF

1.1 Recap: The Homework 7 Picture

Homework 7 built matrix factorization for a tiny 5×4 rating matrix in MATLAB with a pair of nested `for` loops over every observed entry. The approach is conceptually complete: initialize P and Q randomly, compute $e_{ui} = r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i$, apply the SGD updates, and iterate. For a 5×4 matrix this finishes in milliseconds.

1.2 Why That Breaks at Industrial Scale

A realistic recommender has $M \approx 10^8$–10^9 users and $N \approx 10^6$–10^8 items. Three problems arise immediately:

- **Storage.** A dense $M \times N$ matrix is far too large to hold in RAM, on a single disk, or even on a single machine. Nothing you can buy will open it.
- **Sparsity.** Almost every entry is missing. Iterating a nested `for` loop over $M \cdot N$ cells wastes essentially all of its work on zeros.
- **Serial bottleneck.** Even restricting to observed entries, a single-machine SGD loop cannot process billions of updates per epoch within any practical wall time.

1.3 The Shift

The lecture's framing was explicit: "the idea is okay, but the engineering is not feasible." The rest of class introduced the big-data stack — **Hadoop**, **Spark**, and the **PySpark ALS** recommender API — that solves the engineering side so the same algorithm can run over a billion-row rating matrix on a cluster.

2 Big Data: The Four V's

Per the Hadoop slides, roughly 90% of the world's data was produced in just the last two years, and 80% of it is unstructured. A "big data system" is defined by four characteristics:

Volume. Internet and mobile usage dumped unprecedented quantities of data on every industry. Traditional relational databases break well before modern corpora fit.

Variety. Structured SQL tables are no longer the dominant form. Most data now arrives unstructured: text, images, audio, logs, clickstreams, JSON, XML.

Velocity. Users upload hundreds of hours of video to YouTube every minute, send hundreds of thousands of tweets, and transmit hundreds of millions of emails — per minute. The stream never stops.

Veracity. Data streams are noisy and uncertain. Values disagree, sensors drift, users lie. Models have to cope with unreliable inputs.

Traditional single-machine tools cannot address any of these at the scales involved, which is why Google built a new architecture from scratch.

3 Hadoop and HDFS

3.1 Origin and Purpose

Hadoop is an open-source, Java-based framework for distributed storage and processing of very large datasets. It is modeled on the **Google File System (GFS)** and the **MapReduce** programming model Google used to index the web.

3.2 Hadoop Distributed File System (HDFS)

HDFS is Hadoop's storage layer — a UNIX-style, rack-aware filesystem that stripes files across thousands of commodity machines. Key characteristics:

- **Partitioned across hosts.** Files are split into blocks and replicated across many nodes so computation runs close to the data.
- **Fault tolerant.** Block replication lets the cluster survive individual machine failures without losing data.
- **Commodity hardware.** No exotic servers required — scale is achieved by adding cheap boxes.
- **Huge datasets.** Yahoo! historically ran 40,000-server HDFS clusters storing 40 petabytes of data, with a single cluster up to 4,000 machines.
- **Master–slave paradigm.** A name node tracks metadata; data nodes hold blocks.
- **Write-once.** Files are immutable once written — this constraint is what makes replication and fault recovery tractable.

4 MapReduce

4.1 Programming Model

MapReduce is the programming paradigm that originally sat on top of HDFS. It decomposes a job into two phases that each operate on key/value pairs:

1. **Map phase.** Each node receives a partition of the input and emits intermediate key/value pairs. The user-provided `Map()` function defines the transformation. Map input: `list(k1, v1)`; map output: `list(k2, v2)`.
2. **Shuffle.** The framework groups all intermediate values that share the same key and ships them to the reducer responsible for that key.
3. **Reduce phase.** Each reducer receives `(k2, list(v2))` and emits the final `(k3, v3)`. The user-provided `Reduce()` function defines the aggregation.
4. **Final write.** Reduce output is collected and written back to HDFS.

4.2 Why It Scales

The Map step is embarrassingly parallel: every mapper processes its own HDFS block without touching any other. The Shuffle moves only the intermediate key/value pairs, which are usually

much smaller than the raw inputs. The Reduce step runs in parallel over disjoint key ranges. The result is linear scalability across thousands of nodes, fault-tolerant by virtue of HDFS replication.

4.3 Its Limitation — And Why Spark Was Needed

Every MapReduce stage writes its intermediate output back to disk. An iterative algorithm like SGD or ALS — which sweeps the data dozens of times — pays a disk-I/O penalty on every pass. That is the specific pain point Apache Spark was built to fix.

5 Apache Spark

5.1 What It Is

Spark is a fast cluster-computing framework for processing, querying, and analyzing big data. It is **not a replacement for Hadoop** — it runs on top of it, reading from HDFS and reusing Hadoop's cluster resources. Its defining feature is **in-memory computation**: intermediate results stay in RAM across stages instead of being spilled to disk after each one.

- Up to **100× faster** than MapReduce when workloads fit in memory.
- Up to **10× faster** than MapReduce even on disk-bound workloads.
- Originally built at UC Berkeley's AMPLab (2009), donated to Apache and open-sourced in 2010.
- Written primarily in **Scala**, with APIs exposed for Java, Python, and R.

5.2 Python vs. Scala

Scala is Spark's native language and is the fastest option because Spark itself runs on the JVM. Python (PySpark) is slower — it uses process-based executors rather than thread-based ones, and it is not a JVM language, so Python objects must be serialized to cross the boundary. The tradeoff is accessibility: because Python is already the lingua franca of data science, PySpark lets people use Spark without learning a new language. For this class, PySpark is the right choice.

6 Spark Architecture

Every Spark application consists of three pieces:

Driver program. Runs the `main()` function, defines the distributed datasets, and launches parallel operations against them.

SparkContext (`sc`). The driver's handle to the cluster. In the PySpark shell it is pre-created as the variable `sc`. It is through `sc` that you build RDDs, e.g. `sc.textFile("myData.txt")`.

Cluster manager + worker nodes. The cluster manager (Standalone, Mesos, or YARN — Spark is agnostic) allocates executor processes on worker nodes. Each executor runs tasks and caches data in its own local memory.

A Spark program's flow is always the same: the driver creates a `SparkContext`, builds RDDs or DataFrames, and issues parallel operations that the cluster manager schedules as tasks on the executors.

7 RDDs: Spark's Core Abstraction

7.1 Definition

A **Resilient Distributed Dataset (RDD)** is an **immutable, partitioned collection of objects** that Spark distributes across the cluster. Each RDD is split into multiple partitions, and each partition can live on and be computed at a different node. RDDs can hold any Python/Java/Scala object, including user-defined classes.

7.2 Two Ways to Create RDDs

- **Load from external storage:** `lines = sc.textFile("myData.txt")`
- **Parallelize a local collection:** `nums = sc.parallelize([1, 2, 3, 4])`

7.3 Transformations vs. Actions

RDDs expose two kinds of operations — and the distinction is critical:

Transformations construct a *new* RDD from an existing one. They are **lazy**: no computation happens when you call them. Examples: `map()`, `filter()`, `flatMap()`, `distinct()`, `reduceByKey()`.

Actions compute a *result* and either return it to the driver or write it to external storage. They **trigger** all pending transformations in the chain that produced them. Examples: `count()`, `first()`, `collect()`, `reduce()`, `countByKey()`.

7.4 Lazy Evaluation

When you call a transformation, Spark records a *recipe* for computing the resulting RDD rather than computing it on the spot. Nothing runs until an action is called. This matters because Spark can then inspect the whole transformation chain and execute only the parts actually needed.

Lecture's library analogy. If a coworker asks you to "go to the library and bring back the Wikipedia encyclopedia," a naive (eager) worker will haul all 1,000 volumes to the office. A lazy worker waits for the follow-up request — "actually, just the first page" — and then fetches only the first volume. Spark is the lazy worker, and the savings dominate when the dataset is billions of rows but the final answer touches only a handful.

Consequences:

- Spark can fuse adjacent transformations into a single physical pass over the data.
- `sc.textFile()` does *not* read the file immediately. `first()` will read only as many records as it needs.
- By default, an RDD is recomputed each time an action is invoked on it. Use `RDD.persist()` to keep a reused intermediate in memory.

7.5 Typical Spark Program Skeleton

1. Create input RDDs from external data.
2. Transform them with operations like `filter()` and `map()`.

3. Call `persist()` on any intermediate RDD you will reuse.
4. Launch actions like `count()` or `first()` to kick off parallel execution.

8 Common Transformations and Actions

8.1 Element-Wise Transformations

`map(f)` Apply `f` to each element; one input element → one output element.

```
nums   = sc.parallelize([1, 2, 3, 4])
squared = nums.map(lambda x: x * x).collect()   # [1, 4, 9, 16]
```

`filter(f)` Keep only elements where `f` returns true.

```
nums.filter(lambda x: x % 2 == 0).collect()     # [2, 4]
```

`flatMap(f)` Apply `f` returning an iterator, then flatten. One input → zero or more outputs. Contrast with `map`, which preserves the list-of-lists structure.

```
lines = sc.parallelize(["hello world", "hi"])
words = lines.flatMap(lambda line: line.split(" "))   # ["hello","world","hi"]
```

`distinct()` Drop duplicates.

8.2 Actions

`reduce(f)` Combine elements pairwise with a binary function. For `[1,2,3,3]` and `+`, the result is `9`.

`collect()` Pull every element back to the driver as a Python list. **Do not call this on a big RDD** — the full dataset has to fit in the driver's RAM.

`count()` Return the number of elements.

`first()` Return the first element. Cheap under lazy evaluation because Spark only reads enough to produce one row.

9 Pair RDDs (Key/Value)

When each element is a `(key, value)` tuple, Spark unlocks a richer API called a **pair RDD**. Pair RDDs are the workhorse for aggregation jobs.

9.1 Creating a Pair RDD

```
pairs = lines.map(lambda x: (x.split(" ")[0], x))
```

9.2 Useful Pair-RDD Operations

`reduceByKey(f)` Aggregate values that share a key. Output of `[(1,2),(3,4),(3,6)]` with `+`: `[(1,2),(3,10)]`.

`mapValues(f)` Apply `f` to each value, leaving keys untouched.

`values()` / `keys()` Project out one side of the pair.

`countByKey()` Return a dictionary of key → count, materialized to the driver.

`join(other)` Inner-join two pair RDDs on their keys.

9.3 Canonical Example: Word Count

```
rdd    = sc.textFile("myData.txt")
words  = rdd.flatMap(lambda x: x.split(" "))
result = words.map(lambda x: (x, 1)).reduceByKey(lambda a, b: a + b)
```

9.4 Per-Key Average (from the Spark slides)

```
rdd.mapValues(lambda x: (x, 1)) \
   .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1]))
```

This produces `(key, (sum, count))` pairs from which the average is one more division.

10 DataFrames in PySpark

Spark exposes three data representations: RDD, DataFrame, and Dataset. PySpark uses only RDDs and DataFrames (the Dataset API is Scala/Java only).

Why DataFrames are preferred. A DataFrame carries schema metadata — column names, types, nullability — which lets Spark's Catalyst optimizer plan queries intelligently. The result is substantially faster than equivalent RDD code, and the programming model looks familiar to anyone who has used pandas. DataFrames can be created from:

- CSV, JSON, Parquet, XML files.
- RDBMS tables (JDBC), Hive, Cassandra.
- Existing RDDs, by attaching a schema.

Key warning. A PySpark DataFrame *looks* like a pandas DataFrame but is not. A pandas DataFrame has to fit in a single machine's memory. A PySpark DataFrame can represent billions of rows distributed across a cluster, and its operations are lazy. Calling `.show(5)` triggers just enough work to print five rows; calling `.toPandas()` materializes everything and will crash the driver if the dataset is big.

11 PySpark ALS for the Final Project

11.1 Environment Setup on Google Colab

The professor demonstrated running the PySpark recommender live in Google Colab:

1. Open a new Colab notebook and attach a runtime.
2. Install PySpark in the notebook: `!pip install pyspark`.
3. Check the Java version — PySpark requires a matching JVM. Some Spark versions pinned in the sample code do not work with the default Colab Java; installing a compatible JDK and restarting the runtime fixes the mismatch.
4. Upload `trainItem.data` and `testItem.data` (both provided on Canvas) to the Colab session.

The lecture noted that environment debugging will almost certainly be necessary each semester as Colab's defaults drift — students are encouraged to paste errors into Gemini or ChatGPT to resolve version mismatches rather than waiting for office hours.

11.2 Data Format

The training file does *not* store a full rating matrix. Every row is a triple: `userID, itemID, rating`. Zero entries are simply absent. This sparse row-per-observation format is exactly what ALS needs — it scans only the observed ratings rather than iterating across every cell of a dense matrix.

11.3 ALS Pipeline

The reference script distributed with this week's materials (`PySpark Recommendation Code for the Final Project.py`) is a complete end-to-end template. It pulls the following pieces from `pyspark.sql` and `pyspark.ml.recommendation`:

```
from pyspark.sql import SparkSession
from pyspark.sql.types import IntegerType, FloatType
from pyspark.ml.recommendation import ALS
```

The workflow:

1. **Start a SparkSession** with `SparkSession.builder.appName(...).getOrCreate()`. Needed to read CSVs into a DataFrame.
2. **Read** `trainItem.data` into a DataFrame with `spark.read.csv(..., header=False)`. The raw file has no header, so columns come in as `_c0, _c1, _c2`.
3. **Rename columns** to `userID`, `itemID`, `rating` with chained `withColumnRenamed(...)` calls, and **cast** them with `withColumn(..., col.cast(IntegerType()))` / `FloatType()`. ALS will not consume untyped string columns.
4. **Sanity-check** with `training.show(5)`.
5. **Configure the ALS estimator**:

   ```
   als = ALS(
       maxIter=5,
       rank=5,
       regParam=0.01,
       userCol="userID",
       itemCol="itemID",
       ratingCol="rating",
   ```

```
    nonnegative=True,
    implicitPrefs=False,
    coldStartStrategy="drop"
)
```

The knobs correspond directly to the math from Week 10:

- `rank=5` is K — the number of latent factors.
- `maxIter=5` is the number of ALS sweeps.
- `regParam=0.01` is the L2 coefficient λ on $\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2$.
- `nonnegative=True` clamps factors to ≥ 0, which often makes them more interpretable and mimics NMF-style decomposition.
- `implicitPrefs=False` says these are *explicit* ratings (not click counts), so the loss is the standard squared error on observed entries.
- `coldStartStrategy="drop"` tells the model to drop rows whose user or item was never seen in training, instead of emitting `NaN` predictions that would break downstream metrics.

6. **Fit the model**: `model = als.fit(training)`. Under the hood this runs alternating least squares in parallel across the cluster.
7. **Load** `testItem.data` the same way — same column renames, same type casts.
8. **Score the test pairs**: `predictions = model.transform(testing)`. The result is the test DataFrame with an extra `prediction` column.
9. **Write predictions** two ways:
   - `predictions.coalesce(1).write.csv("predictions", header=True)` — the distributed-friendly path, which collapses all partitions into one output file inside a `predictions/` folder.
   - `predictions.toPandas().to_csv("mypredictions.csv", index=False)` — the convenience path that pulls the full result back to the driver as a pandas DataFrame. Safe here because the test set is small, but dangerous at true cluster scale.
10. **Shut down**: `spark.stop()`.

11.4 Why ALS, Not SGD

Week 10's lecture ended with the parallelization argument for ALS: holding P fixed makes every $\mathbf{q}_i$ a standalone ridge-regression problem, and vice versa. That means ALS can update all of Q (or all of P) in parallel across the cluster — which is exactly why `pyspark.ml.recommendation.ALS` is ALS and not SGD. The engineering choice flows directly from the math.

11.5 Data Files Accompanying the Lecture

The `Data/` subfolder distributed with this week's materials contains:

- `trainItem.data` — the training triples (u, i, r) from the Yahoo Music dataset.

- `testItem.data` — the test pairs the model must score.
- `re_trainData2.txt` and `re_u.data` — smaller reshaped variants for local experimentation.

12 Why This Matters for the Ensemble

12.1 Heuristic vs. Collaborative

The heuristic approach used in Homework 4 and the midterm was *user-by-user*: to score Joe on a track, it looked only at Joe's own history. That has a hard failure mode — **cold start**. If Joe has no history, there is nothing to look at and the method is blank.

Matrix factorization is fundamentally different. Every factor in P and Q is learned from *all* users and *all* items at once. Joe's latent vector $\mathbf{p}_{\text{Joe}}$ is shaped by the entire cluster of people whose tastes align with his, and every track's vector $\mathbf{q}_i$ is informed by every user who touched it. There is no user-by-user silo — the whole rating matrix participates in every prediction.

12.2 Diversity for Ensembling

The professor was explicit that the goal is *not* necessarily better accuracy from MF alone. The goal is a prediction that disagrees with the heuristic in useful ways. Because the two approaches draw on completely different signal — one is personal history, the other is global cross-user interaction — their errors are weakly correlated, and averaging them tends to beat either one on its own. That is the setup for the end-of-semester **ensemble** lecture, where every Kaggle submission made during the course (good, bad, mediocre) gets thrown into a blend.

Practical advice that came with this: do not delete your weak submissions. Keep every result, label them, and upload them. The ensemble lecture will use the full collection, and removing the "bad" ones removes information that would otherwise lift the final blended score.

13 Key Reference

HDFS: Hadoop Distributed File System — replicated, rack-aware, write-once block storage across commodity servers.

MapReduce: $(k_1, v_1) \xrightarrow{\text{map}} (k_2, v_2) \xrightarrow{\text{shuffle}} (k_2, \text{list}(v_2)) \xrightarrow{\text{reduce}} (k_3, v_3)$

SparkContext: The driver's connection to the cluster; pre-bound as `sc` in the shell.

RDD: Immutable, partitioned, fault-tolerant distributed collection.

Transformation: Lazy — builds a new RDD (`map`, `filter`, `flatMap`, `reduceByKey`).

Action: Eager — triggers execution (`count`, `first`, `collect`, `reduce`).

Lazy evaluation: No work runs until an action forces it; Spark fuses the transformation chain.

DataFrame: Schema-aware distributed table; preferred over raw RDDs in `pyspark.ml`.

ALS: Alternating Least Squares — the matrix-factorization trainer exposed as `pyspark.ml.recommendation.ALS`.

Rating prediction: $\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$ with $\mathbf{p}_u, \mathbf{q}_i \in \mathbb{R}^K$, learned by ALS on the cluster.